

Valgrind

Laboratórios de Informática III Guião #5-1

Departamento de Informática
Universidade do Minho

Outubro de 2025

Contents

1	Introdução	2
2	Uso da ferramenta	2
3	Valgrind - Memcheck: Erros de memória	2
4	Valgrind - Memcheck: Fugas de memória	3
5	Opções relevantes	4

1 Introdução

A *framework* Valgrind agrega um conjunto de ferramentas para análise de programas. Para programas C em particular, o Valgrind é bastante utilizado para análise de problemas de memória, como *leaks* ou acessos ilegais. Esta análise é feita pela ferramenta `memcheck`, a ferramenta selecionada por defeito pela *framework*.

Este documento irá focar-se apenas na ferramenta `memcheck`. Para mais detalhes sobre as restantes ferramentas, e para uma descrição mais pormenorizada da ferramenta `memcheck` recomendamos a leitura do manual do Valgrind [1].

2 Uso da ferramenta

A plataforma Valgrind não requer o uso de diretivas de programação especiais nem nenhum processo de compilação especial. Para executar a análise de um programa recorrendo à ferramenta Valgrind, basta executar o comando:

```
valgrind [valgrind-options] your-prog [your-prog-options]
```

De notar, que este comando é equivalente a executar o Valgrind com a ferramenta por defeito (`memcheck`):

```
valgrind --tool=memcheck [other-valgrind-options] your-prog [your-prog-options]
```

No entanto, para que o output gerado contenha as referências corretas para ficheiros, funções, linhas, etc., **é necessário compilar o executável com recurso à opção -g**. Os níveis de otimização `-O2` e `-O3` podem levar a ferramenta `memcheck` a gerar erros que na verdade não o são. Desativar otimizações por completo seria a melhor opção para evitar este comportamento, no entanto isso pode tornar a execução da análise extremamente lenta, pelo que o manual do Valgrind recomenda o uso da opção de otimização `-O` (equivalente a `-O1`).

Todas as linhas de output da plataforma Valgrind seguem o formato:

```
1 ==12345== some-message-from-Valgrind
```

Onde `==12345==` indica o número do processo analisado pelo Valgrind. Para efeitos da cadeira de LI3, este número pode ser ignorado.

3 Valgrind - Memcheck: Erros de memória

A ferramenta `Memcheck` consegue captar vários tipos de erros de memória. Consoante o tipo de erro, as mensagens mostradas pelo Valgrind variam, mas o formato geral de output segue o mesmo estilo. Por exemplo:

```
1 ==25832== Invalid read of size 4
2 ==25832==    at 0x8048724: ReSize (bogon.c:45)
3 ==25832==    by 0x80487AF: main (bogon.c:66)
4 ==25832== Address 0xBFFFF74C is not stack'd, malloc'd or free'd
```

No exemplo acima, a mensagem de erro indica que o programa tentou efetuar uma leitura de 4 bytes, na posição `0xBFFFF74C` da memória, que não está alocada na *stack* nem na *heap*, nem foi recentemente libertada. Para além disso, indica onde é que a leitura foi efetuada (linha 45 do ficheiro `bogon.c`).

Olhemos agora para um segundo exemplo:

```
1 ==3004== Invalid read of size 8
2 ==3004==    at 0x10892E: main (test.c:57)
3 ==3004== Address 0x522d648 is 0 bytes after a block of size 8 alloc'd
4 ==3004==    at 0x4C2FB0F: malloc (in /usr/lib/valgrind/vgpreload_memcheck-amd64-linux.so)
5 ==3004==    by 0x108823: get_usernames (test.c:30)
6 ==3004==    by 0x1088DD: main (test.c:54)
```

Neste caso, houve uma tentativa de ler 8 bytes a partir da posição 0x522d648, que está localizada 0 bytes após um bloco de 8 bytes que foi previamente alocado. Ou seja, em `test.c:57` foi alocado um bloco de 8 bytes. Depois, em `test.c:30` o programa tentou ler 8 bytes a partir da posição imediatamente a seguir ao fim desse bloco. O código abaixo geraria um erro semelhante (mas não idêntico):

```
1  ...
2  char line[8];
3  ...
4  line[8]; //A posição 8 deste array de caracteres está fora dos limites do array previamente
5           // definido estando a tentar aceder à posição imediatamente a seguir ao bloco alocado,
6           // i.e., 0 bytes após o bloco.
```

De notar que neste segundo exemplo, o Valgrind retorna informação não só sobre o local onde o acesso ilegal ocorreu, mas também sobre o local onde a variável que foi acedida fora dos seus limites foi inicialmente alocada.

No fim do programa, é mostrado um resumo sobre os erros encontrados:

```
1  ==898499== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Quando o mesmo erro é encontrado múltiplas vezes, apenas é contabilizado uma vez (e.g., um acesso ilegal dentro de um loop).

Os exemplos acima descrevem **acessos ilegais de memória**. A identificação deste tipo de erros pode ajudar a resolver problemas que levam ao crash de programas com mensagens de erro como `segmentation fault`, ou `Command terminated by signal 11`. Para além de acessos ilegais de memória, a ferramenta `memcheck` consegue também identificar: uso de **variáveis não inicializadas**, uso de **valores não inicializados ou não acessíveis em chamadas de sistema**, **freeds ilegais** (e.g., `freeds duplicados`), operações de cópia em que os **locais de origem e destino se sobrepõem**, e operações em que são passados **argumentos que provavelmente estão errados** (e.g., `malloc` com valor negativo como argumento). Para mais informações sobre estes tipos de erros e exemplos de output, consultar a Secção "4.2 Explanation of error messages from Memcheck" do manual de referência [1]. A ferramenta `memcheck` permite também identificar outros tipos de erros que para além de causarem o crash de programas, podem levar a problemas mais difíceis de diagnosticar, como é o caso do uso de variáveis não inicializadas.

4 Valgrind - Memcheck: Fugas de memória

A ferramenta `memcheck` permite também verificar a ocorrência de fugas de memória. Fugas de memória acontecem quando blocos de memória alocados em *heap* não são desalocados antes da conclusão do programa. No fim da execução, um resumo como o abaixo é mostrado:

```
1  ==898499== HEAP SUMMARY:
2  ==898499==    in use at exit: 22,422,087 bytes in 793,472 blocks
3  ==898499== total heap usage: 2,865,887 allocs, 2,072,415 frees, 51,535,906 bytes allocated
```

Este resumo indica-nos, neste caso, que o programa tinha 22 MB de dados alocados no momento de término do programa. Para além disso, podemos observar que há uma discrepância entre o número de blocos alocados e libertados, sendo essa discrepância equivalente ao número de blocos de memória que não foram desalocados. Finalmente, é dada a informação de que, no total, o programa alocou um total de 51MB durante a sua execução.

É possível obter informações adicionais sobre fugas de memória utilizando a opção `--leak-check`. Por defeito, esta opção é definida com o valor `summary`, que exhibe o resumo mostrado abaixo:

```
1  ==898554== LEAK SUMMARY:
2  ==898554==    definitely lost: 5,161,131 bytes in 405,770 blocks
3  ==898554==    indirectly lost: 867,014 bytes in 18,066 blocks
4  ==898554==    possibly lost: 33 bytes in 3 blocks
5  ==898554==    still reachable: 16,393,909 bytes in 369,633 blocks
6  ==898554==    suppressed: 0 bytes in 0 blocks
```

Blocos de memória *heap* são dados como perdidos quando deixam de existir apontadores para esses blocos, passando assim a ser impossível fazer a desalocação desses blocos de memória. Para mais detalhes sobre interpretação de fugas de memória e valores possíveis para a opção `--leak-check`, consultar a Seção "4.2.8 Memory leak detection" do manual de referência da plataforma Valgrind [1].

5 Opções relevantes

Abaixo encontram-se um conjunto de opções que podem ser úteis para o uso da plataforma no contexto do trabalho prático de LI3:

```

1  //General Valgrind options:
2
3  --track-fds=<yes|no|all> [default: no]
4      When enabled, Valgrind will print out a list of open file descriptors on exit or on request,
5      ↪ via the gdbserver monitor command v.info open_fds. Along with each file descriptor is
6      ↪ printed a stack backtrace of where the file was opened and any details relating to the file
7      ↪ descriptor such as the file name or socket details. Use all to include reporting on stdin,
8      ↪ stdout and stderr.
9
10 --log-file=<filename>
11     Specifies that Valgrind should send all of its messages to the specified file. If the file name
12     ↪ is empty, it causes an abort. There are three special format specifiers that can be used in
13     ↪ the file name.
14
15 --num-callers=<number> [default: 12]
16     Specifies the maximum number of entries shown in stack traces that identify program locations.
17
18 --exit-on-first-error=<yes|no> [default: no]
19     If this option is enabled, Valgrind exits on the first error. A nonzero exit value must be
20     ↪ defined using --error-exitcode option. Useful if you are running regression tests or have
21     ↪ some other automated test machinery.
22
23 --max-stackframe=<number> [default: 2000000]
24     The maximum size of a stack frame. [...] You may need to use this option if your program has
25     ↪ large stack-allocated arrays.
26
27 --main-stacksize=<number> [default: use current 'ulimit' value]
28     Specifies the size of the main thread's stack. [...] By default, Valgrind uses the current
29     ↪ "ulimit" value for the stack size, or 16 MB, whichever is lower.
30
31 --redzone-size=<number> [default: depends on the tool]
32     Valgrind's malloc, realloc, etc, add padding blocks before and after each heap block allocated
33     ↪ by the program being run. Such padding blocks are called redzones. The default value for
34     ↪ the redzone size depends on the tool. For example, Memcheck adds and protects a minimum of
35     ↪ 16 bytes before and after each block allocated by the client. This allows it to detect
36     ↪ block underruns or overruns of up to 16 bytes.
37
38 //From memcheck tool:
39
40 --track-origins=<yes|no> [default: no]
41     Controls whether Memcheck tracks the origin of uninitialised values. By default, it does not,
42     ↪ which means that although it can tell you that an uninitialised value is being used in a
43     ↪ dangerous way, it cannot tell you where the uninitialised value came from. This often makes
44     ↪ it difficult to track down the root problem.

```

Nota para verificação de fugas de memória em MacOS

A *framework* Valgrind poderá não ser compatível com sistemas *MacOS* recentes, sobretudo com equipamentos de arquitetura *ARM* (e.g., Mx chips). Como alternativa poderá utilizar a ferramenta `leaks`. Para correr esta ferramenta siga os passos abaixo.

Cada vez que abra um novo terminal terá que ativar opções de *debug*:

```
export MallocStackLogging=1
```

Posteriormente, para detetar *leaks* basta executar o comando:

```
leaks [leaks-options] your-prog
```

Para mais detalhes sobre este programa, pode consultar o manual da ferramenta através do comando bash *manleaks*.

References

- [1] Valgrind Documentation Release 3.20.0 24 Oct 2022
https://valgrind.org/docs/manual/valgrind_manual.pdf